

ACKNOWLEDGEMENT

We express our deep sense of gratitude to our beloved President, Sri. D. Suresh Babu, VNR Vignana Jyothi Institute of Engineering & Technology, for the valuable guidance and for permitting us to carry out this project.

With immense pleasure, we record our deep sense of gratitude to our beloved Principal, Dr B. Chennakesava Rao, for permitting us to carry out this project.

We express our deep sense of gratitude to our beloved Professor Dr T. Sunil Kumar, Professor and Head, Department of CSE-(CyS, DS) and AI&DS, VNR Vignana Jyothi Institute of Engineering & Technology, Hyderabad-500090 for the valuable guidance and suggestions, keen interest and thorough encouragement extended throughout the period of project work.

We take immense pleasure in expressing our deep sense of gratitude to our beloved Guide, Ms Dhanshree Mukund Pande, Assistant Professor in CSE-(CyS, DS) and AI&DS, VNR Vignana Jyothi Institute of Engineering & Technology, Hyderabad, for his/her valuable suggestions and rare insights, for a constant source of encouragement and inspiration throughout my project work.

We express our thanks to all those who contributed to the successful completion of our project work.

C.RITHVIK

25071A6283

TABLE OF CONTENTS

<u>CHAPTERS</u>	<u>PAGE NO</u>
ABSTRACT-----	0
CHAPTER 1 – INTRODUCTION-----	1
CHAPTER 2 – APPROACH / METHODOLOGY-----	3
CHAPTER 2 – CODE SNIPPETS AND DATA SETS-----	4
CHAPTER 3 – TEST CASES/ OUTPUT-----	8
CHAPTER 4 – RESULTS, ANALYSIS AND VALIDATION-----	11
CHAPTER 5 – CONCLUSIONS -----	12
REFERENCES <u>or</u> BIBLIOGRAPHY-----	13
APPENDICES-----	14

ABSTRACT

Managing personal finances requires translating raw, unstructured bank transaction records into clear, actionable insights. This project addresses this need by developing a full-stack, web-based Personal Expense Analyzer and Forecaster. The primary objective is to efficiently parse text-based bank statements, perform robust mathematical analysis, and visualize a user's financial health through a responsive dashboard. The methodology utilizes a custom Python backend leveraging object-oriented programming to model transactions, alongside the NumPy library to compute critical statistics such as totals, means, and standard deviations. Furthermore, a lightweight HTTP server was implemented to provide a RESTful API, bridging the backend analytical pipeline with a vanilla HTML, CSS, and JavaScript frontend. During the development phase, a memory-efficient data pipeline was constructed using Python generators to read files line-by-line, coupled with an interactive user interface capable of handling asynchronous data requests. Key results validated the system's ability to accurately ingest data, handle malformed input lines without crashing, and automatically flag volatile spending categories. Ultimately, the project successfully delivers a highly functional tool for financial tracking, proving the viability of integrating mathematical Python libraries with lightweight web technologies to generate meaningful data analytics.

INTRODUCTION

Background of the Problem

Financial literacy and personal wealth management rely heavily on tracking daily expenses. However, the data exported from modern banks often comes in raw, unstructured formats like CSV or plain text files. While these files contain necessary transactional data, they lack the contextual analysis required for an individual to truly understand their spending habits or forecast future expenses.

Need and Relevance of the Work

There is a distinct need for automated software tools that can quickly ingest raw financial data and apply statistical models to extract meaning. Simply knowing the total amount spent in a month is often insufficient; users need advanced metrics—such as the standard deviation of their purchases—to identify erratic or "highly volatile" spending behaviours that could jeopardize their financial stability.

Problem Statement

The challenge lies in designing a software system capable of reading potentially large raw bank statement files efficiently, mathematically processing that data to find patterns, and presenting the findings in an accessible, web-based graphical user interface without relying on overly complex external frameworks.

Objectives of the Project

The primary objectives of this project are:

- To parse text-based bank statements efficiently using memory-safe Python generators.
- To utilize the NumPy library for computing sums, means, and standard deviations across categorized expenses.
- To design a custom HTTP server that exposes this analytical data via a REST API.
- To develop a responsive, vanilla HTML/CSS/JS frontend dashboard to visualize the data and accept new transaction inputs.

Scope of the Work Carried Out

The scope of this project is restricted to processing standard, three-column transactional data (Date, Category, Amount). It includes the development of the backend parser, the statistical calculation engine, the API layer, and the user interface. The forecasting element is limited to linear annual projections based on current monthly totals.

Organization of the Report

The remainder of this report is organized as follows: Chapter 2 details the software architecture and methodology. Chapter 3 presents the test cases and system outputs. Chapter 4 provides an analysis and validation of the results. Chapter 5 concludes the report and discusses the future scope of the project.

APPROACH / METHODOLOGY

System Architecture

The project adopts a lightweight client-server architecture. To demonstrate core programming competencies, the methodology intentionally avoids heavy frameworks (such as Django or React), relying instead on Python's standard library and vanilla web technologies.

Backend Data Pipeline

The backend processing follows a strict sequential pipeline to ensure data integrity and performance:

- **Memory-Efficient Reading:** The `read_transactions` generator function reads the `bank_statement.txt` file line-by-line, preventing memory overflow issues with exceptionally large files.
- **Object-Oriented Parsing:** The `parse_line` function attempts to convert each raw comma-separated line into a structured `Transaction` object, utilizing exception handling (`try/except`) to gracefully skip malformed lines.
- **Data Aggregation:** A dictionary comprehension is utilized to group the `Transaction` objects by their respective categories (e.g., Groceries, Rent).

Mathematical Analysis using NumPy

Once grouped, the application maps the monetary amounts into 1-Dimensional NumPy arrays. The `analyze_category` function utilizes `np.sum()`, `np.mean()`, and `np.std()` to compute the statistics. A crucial algorithmic step involves comparing the standard deviation against the mean; if the standard deviation exceeds 50% of the mean, the system mathematically flags the category as "Highly Volatile".

API and Frontend Integration

A custom server script (`server.py`) extending `http.server.SimpleHTTPRequestHandler` was developed to route network requests.

- GET requests to `/api/analyze` trigger the data pipeline and return the statistics as a JSON payload.
- POST requests to `/api/transactions` append user-submitted data directly to the text file. The frontend (`app.js`) uses asynchronous `fetch()` API calls to retrieve this JSON and dynamically manipulates the Document Object Model (DOM) to populate the visual dashboard.